Übungsblatt: Spiele

Games.01 - Minimax und Alpha-Beta-Pruning

Die Werte gelten aus Sicht von MAX. Die Kinder werden wie vorgegeben von links nach rechts untersucht. Sobald α ≥ β gilt, wird der restliche Zweig abgeschnitten.

1. Minimax-Bewertungen

Knoten	Berechnung	Wert
B (MIN)	min(8,7,3)	3
E (MAX)	max(9,1,6)	9
F (MAX)	max(2,1)	2
G (MAX)	max(1,6,5,2)	6
C (MIN)	min(9,2,6)	2
D (MIN)	min(2,1,3)	1
A (MAX)	max(3,2,1)	**3**

MAX wählt A → B und garantiert damit den Wert 3.

2. Alpha-Beta von links nach rechts

α gibt die bisher bekannte untere Schranke an, β die obere. Die Werte ändern sich beim Durchlaufen des Baums so:

Knoten	Suchfenster beim Eintritt	Verarbeitung und Änderungen	Rückgabe
A	(−∞,+∞)	nach B: α=3; nach C: α=3; nach D: α=3; β bleibt +∞	3
B	(−∞,+∞)	nach 8: β=8; nach 7: β=7; nach 3: β=3; α bleibt −∞	3
C	(3,+∞)	nach E: β=9; nach F: β=2; nun β≤α, Abbruch vor G	2
E	(3,+∞)	nach 9: α=9; nach 1: α=9; nach 6: α=9; β bleibt +∞	9
F	(3,9)	nach 2: α=3; nach 1: α=3; β bleibt 9	2
G	nicht besucht	kompletter Teilbaum abgeschnitten	kein berechneter Wert
D	(3,+∞)	nach erstem Blatt 2: β=2; nun β≤α, Abbruch	2 als obere Schranke

Zu markierende Kanten: C → G sowie D → Blatt 1 und D → Blatt 3. Der gesamte Teilbaum unter G mit den vier Blättern 1,6,5,2 entfällt ebenfalls. Somit werden sechs der fünfzehn Blätter nicht besucht; neun Blätter werden ausgewertet.

Die besuchten Blätter übernehmen die folgenden α- und β-Werte. Da ihr Wert bereits feststeht, ändern sie die Schranken selbst nicht:

Besuchtes Blatt	Fenster (α,β)
B: 8	$(-\infty,+\infty)$
B: 7	$(-\infty,8)$
B: 3	$(-\infty,7)$
E: 9	$(3,+\infty)$
E: 1	$(9,+\infty)$
E: 6	$(9,+\infty)$
F: 2	(3,9)
F: 1	(3,9)
D: 2	$(3,+\infty)$

Bei den abgeschnittenen Knoten werden keine weiteren Werte berechnet. D würde bei vollständiger Suche den Wert 1 erhalten. Für den Abbruch reicht aber schon das erste Blatt mit Wert 2: D kann damit nicht besser als B mit Wert 3 sein. Bei F bleibt der Rückgabewert 2, auch wenn α schon 3 ist. Die Schranke und der Wert des Knotens sind also auseinanderzuhalten.

3. Bessere Reihenfolge

Ja, zum Beispiel mit dieser Reihenfolge:

• A: **C, B, D**.

• C: **F, E, G**.

• F: 2,1; E: 9,1,6; G: 6,1,5,2.

• B: 3,7,8; D: 1,2,3.

A beginnt mit $(-\infty,+\infty)$. F wird vollständig ausgewertet und liefert 2; damit setzt C $\beta=2$. E liest zuerst 9 und schneidet seine restlichen zwei Blätter ab. G liest zuerst 6 und schneidet seine restlichen drei Blätter ab. C liefert 2, sodass A $\alpha=2$ setzt. B muss alle drei Blätter lesen und liefert 3; A erhöht α auf 3. Bei D genügt das erste Blatt 1, um die beiden übrigen Blätter abzuschneiden.

Damit werden nur **2+1+1+3+1=8 Blätter** ausgewertet. **7 Blätter** werden abgeschnitten, also eines mehr als bei der ursprünglichen Reihenfolge.

Games.02 - Implementierung und Knotenvergleich

Der Code steht in **Spiele.java** und ist in Java geschrieben. Zusätzliche Bibliotheken sind nicht nötig. Zum Kompilieren: javac Spiele.java. Mit java Spiele startet der Vergleich, mit java Spiele --test zusätzlich der Test. Dafür wird ein Java Development Kit (JDK) benötigt.

Das Brett ist ein char-Array mit neun Einträgen in zeilenweiser Reihenfolge. . bedeutet frei. X beginnt und maximiert; O minimiert. Die Nutzenfunktion liefert +1 bei X-Sieg, −1 bei O-Sieg und 0 bei Remis. Bei nichtterminalen Zuständen liefert sie null. Nach einem Sieg wird das Spiel sofort beendet.

search(board, player, false) führt vollständiges Minimax aus. search(board, player, true) aktiviert Alpha-Beta. Beide Varianten verwenden dieselbe Zugreihenfolge 0 bis 8, dieselben Endzustände und dieselbe Bewertung. Bei gleichwertigen Zügen bleibt der erste Zug gewählt. Die Eingabe muss ein legaler Zustand sein, und player muss den Spieler am Zug angeben.

Versuchsaufbau

Als Knoten zählt jeder Aufruf der Suchfunktion, auch die Wurzel und die Endzustände. Wird dieselbe Stellung über eine andere Zugfolge erreicht, zählt sie erneut. Beide Verfahren suchen bis zum Spielende. Bereits berechnete Stellungen werden nicht gespeichert und symmetrische Stellungen nicht zusammengefasst. Der einzige Unterschied beim Vergleich ist das Pruning.

Ausgangszustand	Spieler	Minimax-Knoten	Alpha-Beta-Knoten	Einsparung	Wert	gewähltes Feld
leeres Brett	X	549.946	18.297	96,67 %	0	0
X../.O./..X	O	1.053	318	69,80 %	0	1

Die Schrägstriche trennen Brettzeilen. Feld 0 ist links oben, Feld 1 oben in der Mitte. Die Einsparung ist jeweils (Minimax − Alpha-Beta) / Minimax × 100 %.

Das leere Brett eignet sich für den Vergleich, weil hier noch alle Züge offen sind und am meisten gesucht werden muss. Die zweite Stellung zeigt, wie der Unterschied während einer laufenden Partie ausfällt. Beide Verfahren liefern denselben Zug und den Wert 0, also ein Remis bei optimalem Spiel. Alpha-Beta braucht dafür deutlich weniger Knoten. Wie viel eingespart wird, hängt auch von der Stellung und der Zugreihenfolge ab.

Prüfung

Der mitgelieferte Test geht alle **5.478 legal erreichbaren Brettstellungen** durch. Nach einem Sieg wird nicht weitergespielt. Bei jeder Stellung werden der Spielwert und der gewählte Zug beider Verfahren verglichen. Außerdem wird geprüft, dass Alpha-Beta nicht mehr Knoten besucht als Minimax. Weitere Tests betreffen einen direkten Gewinnzug, das Blockieren eines gegnerischen Gewinns und das leere Brett. Alle Tests sind durchgelaufen, ohne eine Abweichung zu finden.

Games.03 - Negamax

Bei einem Zwei-Personen-Nullsummenspiel gilt: Der Nutzen des einen Spielers ist das Negative des Nutzens des anderen. Für abwechselnde Züge kann deshalb jeder Knoten aus Sicht des jeweils am Zug befindlichen Spielers maximieren:

N(s) = max über Nachfolger t von (−N(t)).

Bei Endzuständen liefert N(s) den Nutzen aus Sicht des Spielers, der in s am Zug wäre. Damit braucht die Suchfunktion keine explizite Fallunterscheidung zwischen MIN und MAX:

```
Negamax(s):
    wenn s terminal:
        return Nutzen aus Sicht des Spielers am Zug in s
    wert = -unendlich
    für jeden legalen Zug a:
        wert = max(wert, -Negamax(Resultat(s,a)))
    return wert
```

Beim Erzeugen des Folgezustands wechselt auch der Spieler am Zug. Die Bewertung muss zu dieser Sichtweise passen: Ein fester Nutzen aus Sicht von X müsste zuerst entsprechend umgerechnet werden.

Beispiel

Die Wurzel R gehört MAX. Ihre beiden Kinder L und M gehören MIN. Unter L stehen die terminalen Werte 3 und 5, unter M die Werte 2 und 9, jeweils aus MAX-Sicht. In den terminalen Zuständen wäre nach dem nächsten Wechsel wieder MAX am Zug.

Knoten	klassisches Minimax (MAX-Sicht)	Negamax (Sicht des Spielers am Zug)
L	min(3,5)=3	max(−3,−5)=−3
M	min(2,9)=2	max(−2,−9)=−2
R	max(3,2)=3	max(−(−3),−(−2))=3

Beide Verfahren wählen L. Die unterschiedlichen Vorzeichen an L und M entstehen allein durch die unterschiedliche Bewertungsperspektive.

Games.04 - Suchtiefe und Evaluierungsfunktion

Die AIMA-Originalaufgabe definiert für **nichtterminale** Zustände:

Eval(s) = 3 X□(s) + X□(s) − 3 O□(s) − O□(s).

X□ zählt Reihen, Spalten und Diagonalen mit genau n X und keinem O. O□ wird entsprechend definiert. Gemischte Linien zählen für keinen der beiden Spieler. Es gibt insgesamt acht Gewinnlinien. Für Endzustände wird der tatsächliche Nutzen U verwendet: X-Sieg +1, O-Sieg −1, Remis 0.

Sechs legale Beispiele

Die drei Brettzeilen sind durch / getrennt; . steht für ein freies Feld. Bei den Endzuständen stehen beide Werte in der Tabelle: das Ergebnis der Formel und der tatsächliche Nutzen. Laut Quelle wird dort der Nutzen verwendet.

Zustand	Brett	X□	X□	O□	O□	rechnerisches Eval	anzuwendende Bewertung
Endzustand: X gewinnt	XXX/OO./...	1	0	0	1	−2	U=+1
Endzustand: O gewinnt	XX./OOO/X..	1	1	1	0	3	U=−1
Endzustand: Remis	XOX/XOO/OXX	0	0	0	0	0	U=0
Zwischenzustand 1	X../.../...	3	0	0	0	3	Eval=3
Zwischenzustand 2	X../.O./...	2	0	3	0	−1	Eval=−1
Zwischenzustand 3	XX./.O./...	1	1	2	0	2	Eval=2

Beispielrechnung für den letzten Zwischenzustand: Die obere Zeile zählt zu X□, die linke Spalte zu X□. Die mittlere Zeile und die Diagonale von rechts oben nach links unten zählen zu O□. Alle übrigen Linien sind leer oder gemischt. Somit Eval=3·1+1−3·0−2=2.

Warum sinnvoll?

Offene Linien zeigen Gewinnmöglichkeiten. Zwei eigene Zeichen in einer offenen Linie werden stärker gewichtet als eines, weil bereits der nächste Zug diese Linie vervollständigen kann. Gegnerische Möglichkeiten verringern den Wert entsprechend. Die Berechnung ist günstig, da nur acht Linien betrachtet werden müssen.

Die Funktion ist trotzdem nur eine Schätzung. Sie berücksichtigt zum Beispiel nicht ausdrücklich, wer als Nächstes am Zug ist. Bei begrenzter Suchtiefe kann sie eine noch offene Stellung bewerten. Ist das Spiel bereits vorbei, wird stattdessen der tatsächliche Nutzen verwendet. Warum das nötig ist, sieht man an den ersten beiden Beispielen: Die Formel allein würde den Gewinner dort falsch einschätzen.

Für eine Suche mit Tiefenbegrenzung muss außerdem die Größenordnung der Werte passen. Eval kann größer als +1 oder kleiner als −1 werden. Ein sicherer Sieg sollte aber besser bewertet werden als jede offene Stellung. Dafür könnte man Siege und Niederlagen mit ±100 bewerten oder die Heuristik auf einen kleineren Wertebereich umrechnen. In der Tabelle stehen die ursprünglichen Nutzenwerte aus der Aufgabe.

Games.05 - Drei Spieler: Max^n

An einem Knoten von Spieler i wird das Kind mit dem größten Wert in Komponente i gewählt. Anschließend wird dessen **gesamtes Tripel** übernommen. Komponenten verschiedener Kinder werden nicht zu einem neuen Tripel zusammengesetzt. Die vier Knoten von Spieler 3 heißen von links nach rechts L□,L□,L□,L□; die beiden Knoten von Spieler 2 heißen P und Q.

Knoten	Vergleich der eigenen Komponente	übernommenes Tripel
L□, Spieler 3	3 > 1	(1,2,3)
L□, Spieler 3	2 > −1	(6,1,2)
L□, Spieler 3	2 > −1	(−1,5,2)
L□, Spieler 3	5 > −1	(5,4,5)
P, Spieler 2	2 > 1	(1,2,3)
Q, Spieler 2	5 > 4	(−1,5,2)
Wurzel, Spieler 1	1 > −1	**(1,2,3)**

Damit lautet der gewählte Pfad: Wurzel → links → links → links. Es gibt bei diesen Entscheidungen keine Gleichstände.

Bonus - Qualität und Zuverlässigkeit generierter Inhalte

Untersuchungsgegenstand und Vorgehen

Als Beispiel dient der mit ChatGPT erzeugte Code Spiele.java. Die Anfrage war, Minimax und Alpha-Beta für Tic-Tac-Toe zu implementieren und die Anzahl der berechneten Knoten zu vergleichen. Zum Abgleich dienen minmax_decision und alpha_beta_search aus der AIMA-Referenzimplementierung. Für die Bewertungsfunktion wird zusätzlich die AIMA-Aufgabe herangezogen. Beide Quellen wurden am 01.11.2026 abgerufen. Neben dem Quellenvergleich wurden die mitgelieferten Tests ausgeführt.

Vergleich

Prüfpunkt	Quellenabgleich / Test	Beurteilung
Minimax-Rekursion	Die AIMA-Referenz alterniert Maximieren und Minimieren und beendet die Suche an terminalen Zuständen. Dies tut auch der erzeugte Code.	Für das modellierte Spiel korrekt.
Alpha-Beta	AIMA beendet MAX-Zweige bei $v \geq \beta$ und MIN-Zweige bei $v \leq \alpha$. Im erzeugten Code wird nach dem Schrankenupdate $\alpha \geq \beta$ geprüft, was denselben Abbruch bewirkt.	Korrekte Schrankenführung.
Bewertungsperspektive	AIMA bewertet relativ zum Wurzelspieler; der erzeugte Code bewertet durchgehend relativ zu X und lässt O minimieren.	Abweichende, konsistente Konvention; O am Zug wurde mitgeprüft.
Übereinstimmung beider Verfahren	Für alle 5.478 legal erreichbaren Stellungen wurden Werte und gewählte Züge verglichen; zusätzlich Gewinn- und Blockierfälle.	Keine Abweichung gefunden. Ein gemeinsamer Fehler beider Verfahren wäre durch den Vergleich allein allerdings nicht ausgeschlossen.
Leistungsangaben	549.946 gegenüber 18.297 Knoten beim leeren Brett wurden tatsächlich ausgegeben. Zählweise und Reihenfolge sind dokumentiert.	Nachvollziehbares Messergebnis für genau dieses Szenario; keine allgemeingültige Quote.
Heuristik und Endzustände	Die AIMA-Aufgabe trennt Eval für Zwischenstände von U für Endzustände. Der rohe Ausdruck kann selbst bei einem X-Sieg negativ sein.	Die Trennung ist notwendig; die pauschale Verwendung der Heuristik an allen Blättern wäre falsch.

Grenzen und Fazit

Bei den geprüften Algorithmen und Messergebnissen wurden keine Fehler oder erfundenen Angaben gefunden. Es fehlen aber eine Prüfung beliebiger Benutzereingaben und eine Bedienoberfläche. Der Code erwartet eine legale Brettstellung und den richtigen Spieler am Zug. Für den geforderten Vergleich reicht das aus; für ein Spiel mit freien Eingaben müsste es ergänzt werden.

Das Ergebnis spricht dafür, dass der erzeugte Code die Aufgabe korrekt löst. Nur die Ausgabe von ChatGPT zu lesen, wäre dafür aber zu wenig gewesen. Erst der Vergleich mit der Quelle und die Tests machen die Ergebnisse überprüfbar. Besonders bei der Bewertung von Endzuständen und beim Zählen der Knoten können kleine Unterschiede zu falschen Aussagen führen. Daraus lässt sich nicht ableiten, dass andere Antworten von ChatGPT ebenfalls fehlerfrei sind.